Model Context Protocol (MCP)

Extending AI Agents with Tools, Resources & Prompts

Module 3: Agentic AI & Deployment • Section 11.1

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover

1

Why MCP?

The problem MCP solves for AI applications

2

Architecture

Hosts, Clients, Servers & two-layer design

3

Server Primitives

Tools, Resources & Prompts explained

4

Client Features

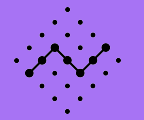
Elicitation, Roots & Sampling

5

Best Practices

Versioning & progressive discovery

Why MCP?
The Problem It Solves



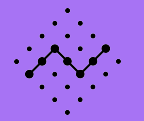
Why MCP? — The Problem It Solves

MCP (Model Context Protocol): A standard that lets AI applications connect to data sources, tools, and workflows — enabling them to access key information and perform real-world tasks.

Without MCP — Isolated AI	With MCP — Connected AI
✗ AI apps are isolated — no access to files, DBs, or APIs	✓ Standard protocol for AI ↔ tool communication
✗ Each integration is a custom one-off build	✓ Connect once, reuse across any MCP-compatible host
✗ No standard way to give LLMs external context	✓ AI can search, write files, query DBs, call APIs
✗ Maintenance nightmare as tools multiply	✓ Community ecosystem of ready-made MCP servers

MCP is to AI agents what HTTP is to the web — a universal communication standard.

MCP Architecture
Hosts, Clients & Servers



Key Participants

Three roles: Every MCP interaction involves a **Host** (the AI app), a **Client** (the protocol handler inside the host), and a **Server** (the capability provider).

1

MCP Host

- The AI application (e.g., Claude Desktop, custom app)
- Manages user interaction & session
- Hosts one or more MCP clients
- Controls permissions & security policies

2

MCP Client

- Lives inside the host application
- Speaks the MCP protocol on behalf of the host
- Connects to one MCP server per instance
- Manages lifecycle & capability negotiation

3

MCP Server

- Exposes tools, resources, and prompts
- Local (same machine, stdio) or Remote (HTTP/SSE)
- Stateless or stateful depending on transport
- Community-built or custom server

Two-Layer Architecture

MCP separates **what** is communicated (Data Layer) from **how** it travels (Transport Layer) — enabling both local and remote deployments with the same protocol.

Data Layer (Inner) — JSON-RPC 2.0

1. Lifecycle Management
Connection init, capability negotiation, termination

2. Server Features
Tools (actions), Resources (context), Prompts (templates)

3. Client Features
Sampling, elicitation, and logging capabilities

4. Utility Features
Notifications and progress tracking for long operations

Transport Layer (Outer) — Communication

Stdio Transport

- Uses standard input/output streams
- Local processes on same machine only
- Zero network overhead
- Optimal performance for local servers

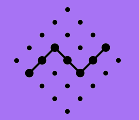
Streamable HTTP Transport

- HTTP POST + optional Server-Sent Events
- Enables remote server communication
- Supports bearer tokens, API keys, OAuth
- MCP recommends OAuth for auth tokens

Data Layer = what is exchanged | Transport Layer = how it travels. Same protocol, flexible delivery.

Server Primitives

Tools · Resources · Prompts



The Three Server Primitives

Servers expose three types of capabilities — each with a distinct role and control model:

Feature	Explanation	Examples	Who Controls It
Tools	Functions the LLM can actively call and decides when to use, based on user requests. Can write to DBs, call APIs, modify files, or trigger logic.	Search flights Send messages Create calendar events	Model
Resources	Passive read-only data sources that provide contextual information — file contents, DB schemas, API responses, documentation.	Retrieve documents Access knowledge bases Read calendars	Application
Prompts	Pre-built instruction templates that tell the model how to work with specific tools and resources for a given domain.	Plan a vacation Summarize meetings Draft an email	User

Tools = Model-driven actions | Resources = App-driven context | Prompts = User-driven templates

1 Tools — Executable Functions

Tools are functions exposed by MCP servers that the AI model can invoke to perform real-world actions — they are model-controlled.

```
{
  name: "searchFlights",
  description: "Search for available flights",
  inputSchema: {
    type: "object",
    properties: {
      origin: {
        type: "string",
        description: "Departure city"
      },
      destination: {
        type: "string",
        description: "Arrival city"
      },
      date: {
        type: "string",
        format: "date",
        description: "Travel date"
      }
    },
    required: ["origin","destination","date"]
  }
}
```

Chained Tool Call Example

1 **searchFlights()**

```
origin:"NYC", destination:"Barcelona",
date:"2024-06-15"
```

↓

2 **createCalendarEvent()**

```
title:"Barcelona Trip",
startDate:"2024-06-15"
```

↓

3 **sendEmail()**

```
to:"team@work.com",
subject:"Out of Office"
```

One user intent → multiple tool calls chained automatically by the model.

The model decides when and which tools to call — the server just exposes what's available.

2 Resources — Data Sources

Resources are passive, read-only data sources that give AI applications contextual information — they are application-controlled, not model-controlled.

Two Resource Types

Direct Resources — Fixed URIs

```
calendar://events/2024
→ returns calendar availability for 2024
```

Resource Templates — Dynamic URIs

```
travel://activities/{city}/{category}
travel://activities/barcelona/museums
→ returns all museums in Barcelona
```

Protocol Operations

Method	Purpose	Returns
`resources/list`	List available direct resources	Array of resource descriptors
`resources/templates/list`	Discover resource templates	Array of template definitions
`resources/read`	Retrieve resource contents	Resource data with metadata
`resources/subscribe`	Monitor resource changes	Subscription confirmation

Parameter Completion: Type "Par" → suggests Paris, Park City — autocomplete for dynamic URI params.

Resources give the AI passive read-only context — the application decides what to expose and when.

3 Prompts — Reusable Templates

Prompts are parameterized instruction templates that MCP server authors provide — guiding the model on how to best use the server's tools and resources.

```
{
  "name": "plan-vacation",
  "title": "Plan a vacation",
  "description": "Guide through vacation
    planning process",
  "arguments": [
    {
      "name": "destination",
      "type": "string",
      "required": true
    },
    {
      "name": "duration",
      "type": "number",
      "description": "days"
    },
    {
      "name": "budget",
      "type": "number",
      "required": false
    },}
```

Protocol Operations

Method	Purpose	Returns
`prompts/list`	Discover available prompts	Array of prompt descriptors
`prompts/get`	Retrieve prompt details	Full prompt definition with arguments

Who Initiates Each Primitive?

⚙️ **Tools:** → Model initiates

📁 **Resources:** → Application initiates

💬 **Prompts:** → User initiates

Prompts are user-initiated — unlike tools (model) or resources (application).

Client-Side Features

Elicitation · Roots · Sampling

Client-Side Features & Scopes

The MCP client exposes three features back to the server — allowing servers to request human input, filesystem access, and LLM completions.

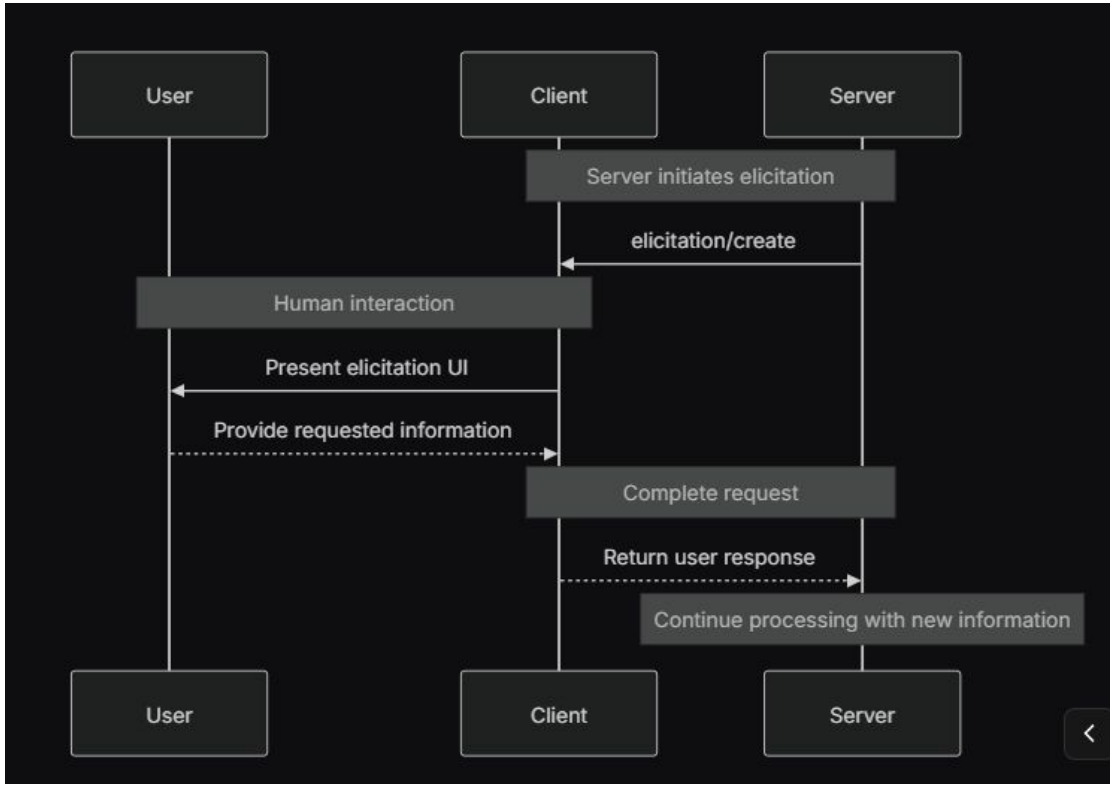
Feature	Explanation	Example
Elicitation	Servers can request specific information from users during interactions — a structured way to gather input on demand when data is missing.	A travel booking server asks for seat preferences, room type, and insurance choice before finalising a booking.
Roots	Clients specify which directories servers should focus on — communicating intended filesystem scope through a coordination mechanism.	A server for booking travel is given access to a specific directory from which it can read a user's calendar.
Sampling	Servers request LLM completions through the client — enabling agentic workflows while keeping the client in complete control of permissions and security.	A travel server sends a list of flights to an LLM and requests that the LLM pick the best flight for the user.

Elicitation = human input | Roots = filesystem scope | Sampling = LLM-as-a-service for servers

Elicitation — Pausing for Human Input

Elicitation: When data or input is missing, the MCP server pauses and requests structured input from the user through an elicitation UI — before continuing the task.

```
{
  method: "elicitation/create",
  params: {
    message: "Confirm Barcelona
      booking details:",
    schema: {
      type: "object",
      properties: {
        confirmBooking: {
          type: "boolean"
        },
        seatPreference: {
          enum: ["window","aisle",
            "no preference"]
        },
        roomType: {
          enum: ["sea view",
            "city view",
            "garden view"]
        },
        travelInsurance: {
          type: "boolean",
          default: false
        }
```



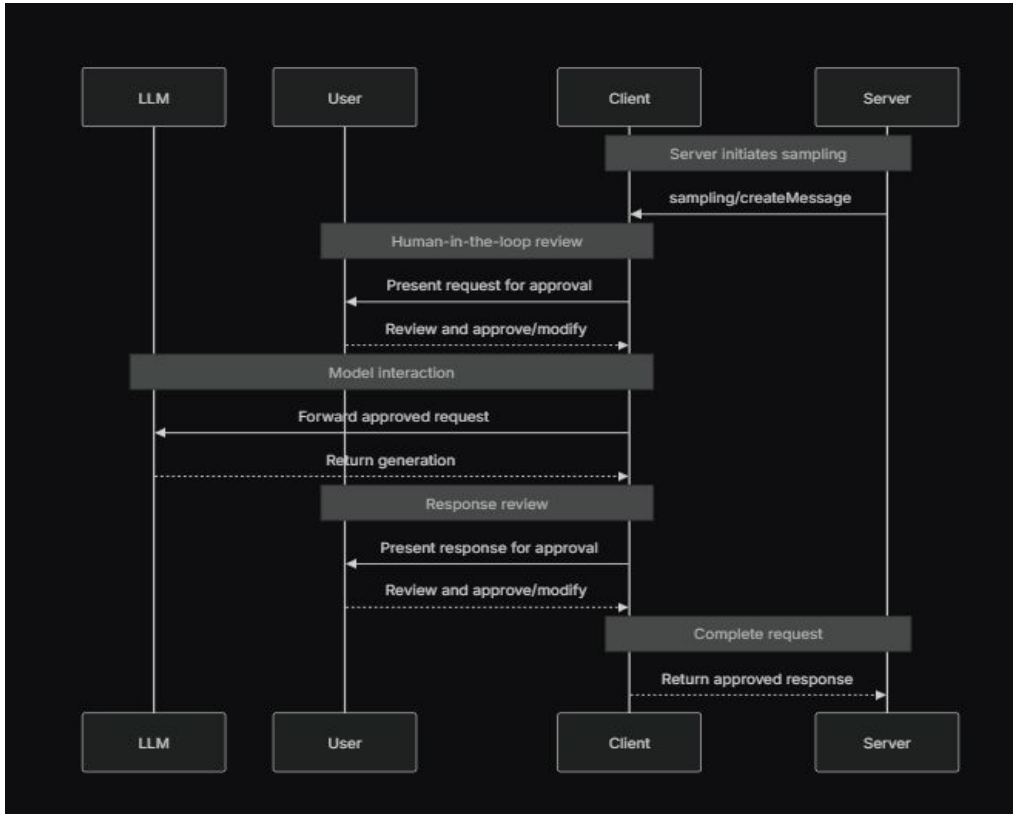
Elicitation keeps humans in the loop — the server cannot proceed until the user provides the required input.

Sampling — LLM-Powered Server Decisions

Sampling lets MCP servers request LLM completions through the client — keeping servers model-independent while gaining AI reasoning capabilities without embedding an LLM SDK.

Standard Flow vs Sampling Flow

	Standard Flow	Sampling Flow
The Setup	Chef brings 100 cookbooks to the kitchen table and reads them all	Chef tells the Intern: 'Bring me the best 3 pages on pasta from the library'
The Result	Table covered in 100 books — no room to actually cook	Intern returns with 3 neat pages; table is clean and ready for work
The Cost	You pay the Head Chef to read 1,000 pages of noise	You pay the Intern a small fee; the Chef only reads the signal



Sampling = model-independent server design + human approval control + cost efficiency.

Roots — Scoping Filesystem Access

Roots define the filesystem boundaries within which an MCP server is permitted to operate — the client communicates scope to the server through root URIs.

```
{
  "uri": "file:///Users/agent/travel-planning",
  "name": "Travel Planning Workspace"
}
```

Manual Root Configuration

User explicitly sets which directories the server can access

Automatic Root Detection

Client infers roots from project context, open folders, or workspace settings

Why Roots Matter

🔒 **Security:** Server cannot access files outside declared roots

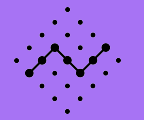
🎯 **Scope Clarity:** Server knows exactly where to look — no ambiguity

📋 **Auditability:** Every file access is within a known, declared boundary

🗂️ **Multi-project Support:** Different roots per server instance — clean separation

Roots give the human operator fine-grained control over what filesystem context each server can see.

MCP Best Practices & Versioning



MCP Client Best Practices

Building robust MCP integrations requires careful tool management, versioning discipline, and security-first design.

Progressive Tool Discovery

→ Maintain a local registry of available tools

→ Don't load all tools upfront — discover on demand

→ Cache tool schemas to reduce network round-trips

→ Refresh registry when server capabilities change

→ Present tools contextually based on user intent

Versioning & Capability Negotiation

Revisions: Servers increment revision numbers when capabilities change; clients check compatibility before use

Negotiation: During lifecycle init, client and server exchange capability manifests; only mutually supported features are used

Graceful Degradation: Never assume a tool exists — always check capability negotiation result

No Breaking Changes: Clients using older capabilities must continue to work after server updates

Treat MCP servers like APIs — version them, document them, and never break existing clients without negotiation.

Key Takeaways — Model Context Protocol

1. **MCP standardizes tool integration:** One protocol for connecting AI apps to any data source, tool, or workflow — replaces custom one-off integrations.
2. **Two-layer design separates concerns:** Data Layer (what is exchanged) + Transport Layer (how it travels) — stdio for local, HTTP/SSE for remote.
3. **Three server primitives:** Tools (model-controlled actions), Resources (app-controlled context), Prompts (user-controlled templates).
4. **Three client-side features:** Elicitation (structured human input), Roots (filesystem scope), Sampling (LLM completions through the client).
5. **Best practices keep integrations robust:** Progressive tool discovery, capability negotiation, and versioning protect against breaking changes.

MCP turns isolated AI models into connected agents — capable of acting on the world through a universal, secure, and extensible protocol.

Advanced Multi-Agent Architectures

Supervisor Patterns, Hierarchies & Communication Protocols

Module 11 / Section 11.2

Nunnari Academy

Generative AI & Agentic AI Program

What We'll Cover

1

Supervisor-Worker Patterns

Hub-and-spoke control: one LLM supervisor routes many specialized workers

2

Hierarchical Agent Organizations

Nested supervisors: domain teams within teams — containing complexity

3

Agent Communication Protocols

Four protocols: how agents share data, messages, functions, and control

4

Choosing the Right Pattern

Matching architecture to latency, coupling, and debuggability requirements

5

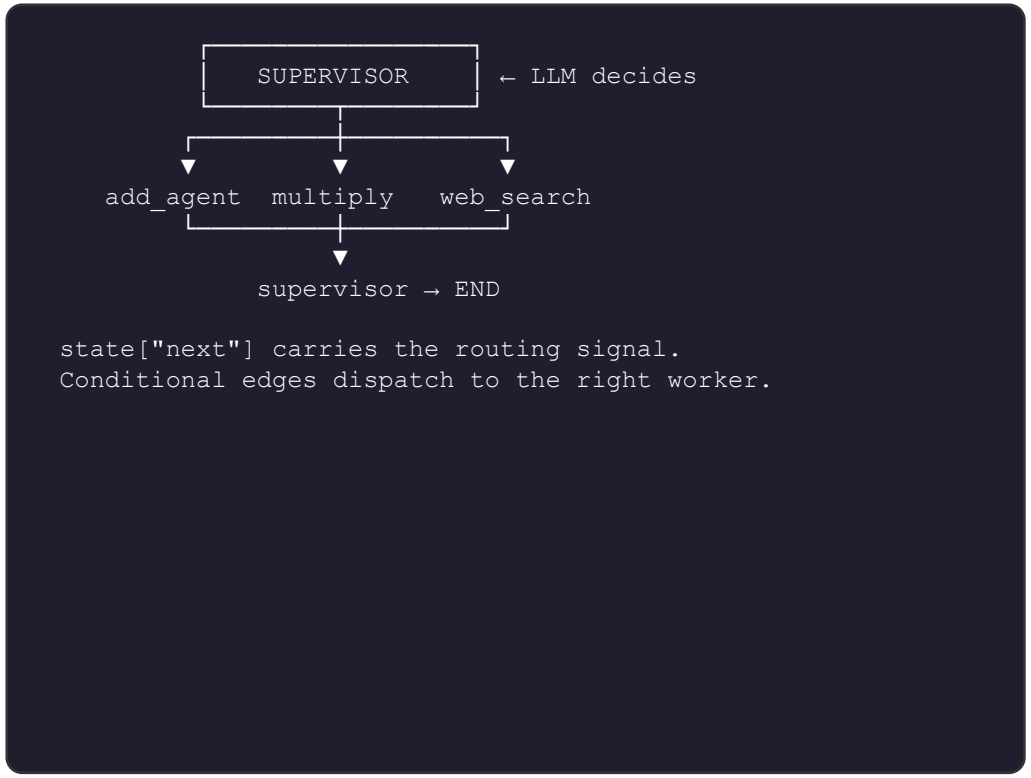
Hands-On Notebooks

Three supervisor patterns + hierarchy graph + all four communication protocols

Supervisor-Worker Patterns

1 Supervisor-Worker Pattern

Supervisor-Worker: A single LLM supervisor reads the task, decides which worker to call next, and routes back to itself until complete.



Routing owner: Supervisor LLM — not hardcoded if/else

Workers: Do one job and return to supervisor

Signal: state["next"] carries the dispatch target

Graph wiring: Conditional edges read next and route

Key trait: The LLM, not developer code, controls routing

The defining trait: routing decisions live inside the LLM, not in developer code.

Pattern 1 — Star (Single-Step Routing)

Star Pattern: Supervisor picks one worker per task and routes directly to END. One task → one worker → done.

```
class State(TypedDict):
    messages: list
    next: str
    result: str

def supervisor(state):
    prompt = f"""Workers: add, multiply,
web_search, END
Task: {state["messages"][-1]}
Reply with ONE word."""
    response = llm.invoke(prompt)
    return {"next": response.content.strip()}

def route(state):
    return {
      "add":        "add_agent",
      "multiply":   "multiply_agent",
      "web_search": "web_search_agent"
    }.get(state["next"], END)
```

Aspect	Behavior
Routing	Supervisor → one worker → END
LLM calls	1 supervisor + 1 worker
State fields	messages, next, result
Best for	Single-intent tasks
Weakness	Cannot chain steps

Example run:
"What is 12 + 45?"
→ supervisor routes to add_agent
→ result: 57.0 → END

Star pattern: one routing call, one worker call, done. Ideal for single-intent queries.

2 Pattern 2 — Sequential (Multi-Step)

Sequential Pattern: Supervisor tracks completed steps in steps_taken and routes to the next logical worker, enabling multi-step execution.

```
class State(TypedDict):
    messages: list
    next: str
    result: str
    steps_taken: list  # ← NEW: tracks history

def supervisor(state):
    steps = state.get("steps_taken", [])
    prompt = f"""Task: {state["messages"][-1]}
Steps completed: {steps}
Workers: add, multiply, web_search, END
What to do NEXT? ONE word."""
    # Supervisor sees history → avoids loops
    ...
```

Key difference: steps_taken field records every completed step

Loop prevention: Supervisor reads history before deciding next

LLM calls: N supervisor calls + N worker calls

State grows: steps_taken accumulates across turns

Best for: Chained tasks: compute, transform, verify

Weakness: More LLM calls than star; not parallel

steps_taken is the memory that prevents infinite loops — supervisor always knows what's done.

3 Pattern 3 — Parallel (Fan-Out & Aggregate)

Parallel Pattern: All workers dispatched simultaneously via Send(). An Annotated reducer merges results; an aggregator node produces the final output.

```
class State(TypedDict):
    messages: list
    results: Annotated[list, operator.add]
    #   ↑ reducer: MERGES parallel outputs
    #     instead of overwriting

def fan_out(state):
    return [        # all 3 fire at once
        Send("add_agent",       state),
        Send("multiply_agent",  state),
        Send("web_search_agent",state),
    ]

def aggregator(state):
    summary = "\n".join(state["results"])
    return {"results": [summary]}
```

Send() API

LangGraph's fan-out primitive dispatches to multiple nodes in a single step — true concurrency.

Annotated Reducer

operator.add appends each worker's result rather than overwriting — safe for parallel writes.

Aggregator Node

Runs after all workers complete. Collects merged results and produces the final consolidated output.

No Supervisor LLM

Fan-out is a pure graph function — zero LLM calls for routing. Workers run without supervision.

LangGraph's Send() enables true parallelism — workers run concurrently, not in sequence.

Supervisor Pattern Comparison

All three share the same supervisor node structure — differences lie in state design and graph wiring.

Aspect	Star	Sequential	Parallel
Steps	Single	Multi	All at once
Key state field	next, result	+ steps_taken	Annotated reducer
LLM routing calls	1 supervisor	N supervisors	0 (fan-out fn)
Parallelism	No	No	Yes — native
Best for	Simple routing	Chained tasks	Independent subtasks
Weakness	Can't chain	More LLM calls	No conditional logic

State design IS the architecture — add one field and you unlock a completely different pattern.

Hierarchical Agent Organizations

2 Hierarchical Agent Organizations

Hierarchy: A top-level supervisor routes to domain sub-supervisors. Each sub-supervisor manages its own team — building a tree of agents.

```
          TOP SUPERVISOR
         /       |       \
        ▼        ▼        ▼
  Arithmetic  Logic    WebSearch
  Supervisor Supervisor Supervisor
    /    \     /    \     /      \
  add  mult eval compare search summarize

Top supervisor knows TEAM names only.
Sub-supervisors know WORKER names only.
```

```
# Sub-graph registered as a single node:
builder.add_node("arithmetic", arithmetic_graph)
builder.add_node("logic",      logic_graph)
builder.add_node("websearch",  websearch_graph)
```

Scalability: One supervisor can't manage dozens of specialized workers

Encapsulation: Domain knowledge stays inside the sub-supervisor — not in top-level prompts

Extensibility: Add a new domain = one new sub-graph, one new node in top-level graph

Isolation: WebSearch team can be sequential internally — invisible to top supervisor

Reusability: Sub-graphs are compiled LangGraphs that can be reused across projects

Hierarchy contains complexity — the top supervisor only knows domain names, not individual workers.

2 Three-Level Graph: Top → Sub-Supervisor → Leaf

Key insight: Each sub-graph is a compiled LangGraph — a complete agent. The top supervisor treats an entire team as a single node.

```
# TOP SUPERVISOR
def top_supervisor(state):
    prompt = """Teams: arithmetic, logic, websearch
Task: {state["messages"][-1]}
Reply ONE word (team name)."""
    # Routes to a TEAM, not a worker

# Graph wiring:
builder.add_node("arithmetic", arith_graph)
builder.add_node("logic",      logic_graph)
builder.add_node("websearch",  web_graph)
builder.set_entry_point("top_supervisor")
builder.add_conditional_edges(
    "top_supervisor", route_team, {...})
```

```
# WEBSEARCH SUB-SUPERVISOR
# Internally a 2-step pipeline:
def websearch_supervisor(state):
    result = state.get("result","")
    prev   = state.get("next","")
    if not result:   next_step = "search"
    elif prev == "search": next_step = "summarize"
    else:            next_step = "end"
    # search → summarize → end (sequential inside)
```

L1 Top Supervisor

Routes by domain name only. Knows: arithmetic, logic, websearch. Prompt stays simple.

L2 Sub-Supervisor

Routes within its domain. WebSearch supervisor runs search → summarize pipeline internally.

L3 Leaf Workers

Do exactly one job: add, multiply, evaluate, compare, search, or summarize.

The WebSearch team runs search → summarize internally — a sequential pattern inside a hierarchy.

Agent Communication Protocols - (Internal)

Agent Communication — Four Protocols

How agents share data determines latency, flexibility, and debuggability.

P1 — Shared State

- Agents read/write named fields in a shared TypedDict
- Supervisor writes next; workers write result
- Workers parse strings to find their numeric inputs
- Fast, direct — no message overhead

Best for: Simple, single-step tasks

P2 — Message Passing

- Agents append messages to a growing list
- Full conversation history = full audit trail
- Every routing decision is a recorded message
- History grows unboundedly → context limit risk

Best for: Transparency, debugging

P3 — Tool Calls

- Workers registered as @tool with typed schemas
- LLM's function-calling IS the routing mechanism
- Structured arguments — no string parsing
- Supports parallel tool calls natively

Best for: Atomic ops, API wrappers

P4 — Direct Handoff

- Each agent hardcodes its successor's name
- No supervisor — agents chain themselves
- Zero supervisor round-trip latency
- Tight coupling: inserting agents requires code changes

Best for: Fixed linear pipelines

Same LangGraph skeleton — swapping the communication protocol changes the system's character entirely.

Protocols 1 & 2 — Shared State vs Message Passing

P1 Shared State

```
class State(TypedDict):
    messages: list    # task input
    next: str         # supervisor writes here
    result: str       # worker writes here

# Worker reads task via STRING PARSING:
def add_agent(state):
    text = state["messages"][-1]
    nums = re.findall(r"\d+\.?\d*", text)
    return {"result": str(float(nums[0])
                          + float(nums[1]))}
```

Channel: Named dict fields (next, result)

Worker reads via: String parsing — fragile re.findall()

Audit trail: None — fields overwritten each turn

Best for: Simple, single-step tasks

P2 Message Passing

```
# The messages list IS the channel.
# APPEND only — never overwrite.

# After one task: state["messages"] =
[
  HumanMessage("What is 5 + 3?"),
  AIMessage("Routing to add_agent"),
  AIMessage("Add result: 8"),
  AIMessage("Task complete"),
]
# Every routing decision is recorded.
```

Strength: Full audit trail — every decision is a message

Weakness: History grows unboundedly → context overflow

Growth rate: Task 1: 4 msgs → Task 20: ~80 msgs

Best for: Short-lived tasks, debugging, transparency

P1: fast, direct, fragile. P2: transparent, auditable, but grows without bound.

P3 Protocol 3 — Tool Calls (Function Calling)

Tool Calls: Workers are Python functions decorated with @tool. The LLM's built-in function-calling IS the routing mechanism — no if/else needed.

```
@tool
def multiply(a: float, b: float) -> float:
    """Multiply two numbers."""
    return a * b

@tool
def divide(a: float, b: float) -> float:
    """Divide a by b."""
    return a / b

# LLM sees typed JSON schemas — not strings.
llm_with_tools = llm.bind_tools([multiply,
                                 divide])
# Graph: supervisor → tools → supervisor → END
```

```
# "Multiply 8 by 6 and divide the result by 2"
# LLM step 1: multiply(a=8, b=6)  → ToolMessage: 48
# LLM step 2: divide(a=48, b=2)   → ToolMessage: 24
# LLM step 3: Final answer: 24    (no prompt change)
```

① **Structured Inputs**

multiply(a=8, b=6) — typed args enforced by schema. No re.findall() string parsing.

② **Multi-Step is Free**

LLM chains tool calls naturally. No supervisor prompt change needed for chained tasks.

③ **Parallel Calls**

LLM can emit multiple tool calls in one response. ToolNode handles them automatically.

Weakness: Workers are plain functions — no internal state, no retry logic, no subgraph.

Tool Calls: structured inputs + free multi-step + native parallelism — at the cost of stateless workers.

P4 Protocol 4 — Direct Handoff

Direct Handoff: Agents chain themselves — each agent hardcodes its successor. No supervisor. Routing is a pure passthrough of state["next"].

```
# Pipeline: outline → draft → refine → END

def outline_agent(state):
    # ... generate 3 key points ...
    return {..., "next": "draft_agent"}
              # ↑ HARDCODED successor name

def draft_agent(state):
    # ... write paragraphs ...
    return {..., "next": "refine_agent"}

def refine_agent(state):
    # ... polish tone ...
    return {..., "next": "END"}

# Route function is trivial — pure passthrough:
def route_next(state):
    return state["next"]    # no logic here
```

```
# Speed comparison for 3-agent pipeline:
# Protocol 2: outline→sup→draft→sup→refine→sup→END
#             6 hops, 3 supervisor LLM calls
#
# Protocol 4: outline → draft → refine → END
#             3 hops,  0 supervisor LLM calls
```

Aspect	P4 Behavior
Routing	Each agent hardcodes next
Supervisor	None — agents decide
Latency	Lowest: no round-trips
Coupling	Tight: agents know each other
Flexibility	Low: inserts need multi-file edits
Best for	Fixed pipelines, content chains

Direct Handoff: fastest pipeline pattern — zero supervisor overhead — but tight coupling is the price.

All Four Protocols — Side by Side

Aspect	P1: Shared State	P2: Message Passing	P3: Tool Calls	P4: Direct Handoff
Routing	Supervisor LLM	Supervisor LLM	LLM tool-call	Each agent hardcodes
Worker reads via	String parsing	Message history	Typed arguments	Message history
Audit trail	None	Full	Partial	None
Latency	Medium	Medium	Medium	Lowest
Parallelism	No	No	Yes — native	No
Coupling	Low	Low	Low	High
Best for	Simple routing	Debugging	Atomic ops	Fixed pipelines

No protocol is universally best — choose based on whether you need flexibility, transparency, structured inputs, or raw speed.

Thank You!

Nunnari Academy
Generative AI & Agentic AI Program